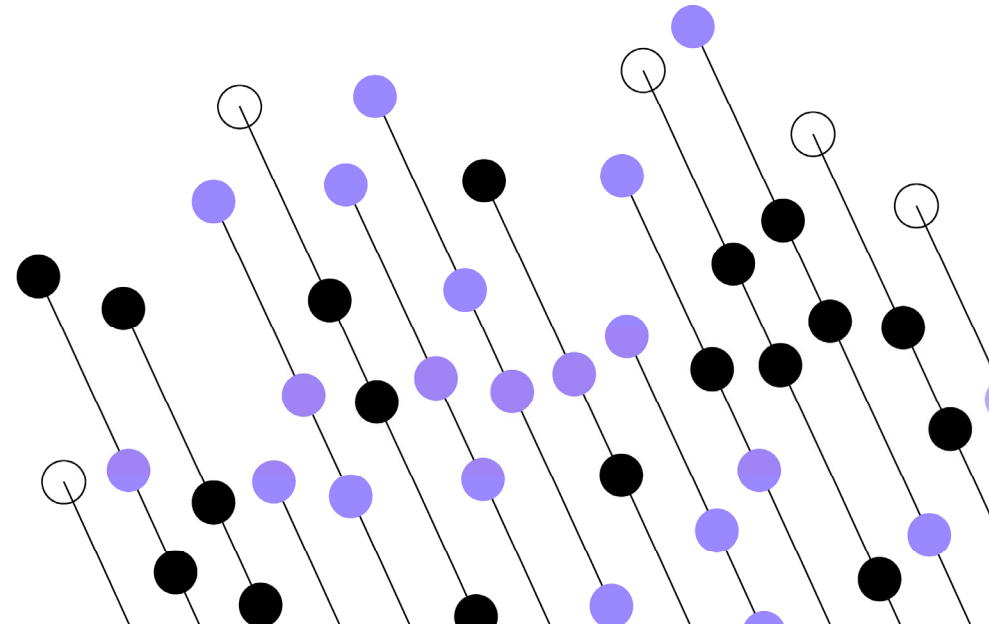
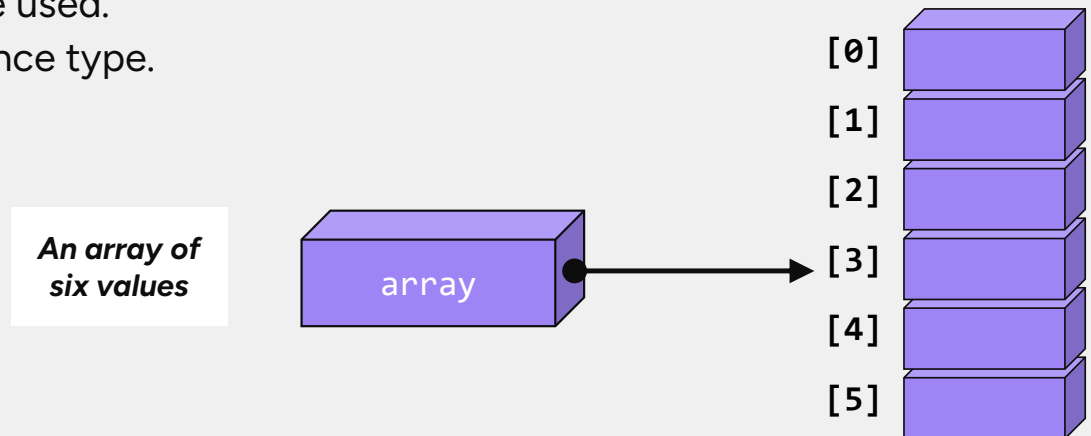


Arrays and Loops



What is an array?

- **An array can store a collection of variables all of the same type:**
 - Each array element can hold a single item (value / reference type).
 - Array elements are accessed by index number, e.g., names[3].
- **Arrays are objects:**
 - They must be created before they can be used.
 - An array variable (of any type) is a reference type.



Introduction to arrays

- An array is a fixed-size collection of elements of the same data type.

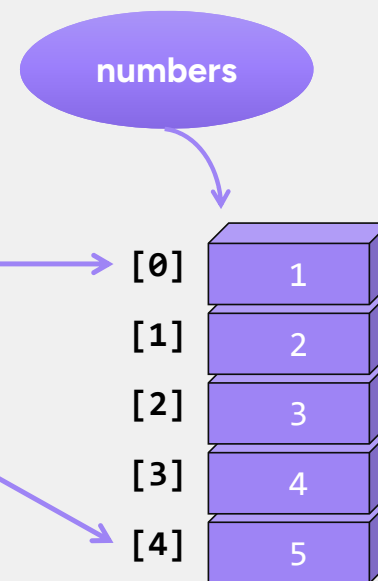
```
int[] numbers = {1,2,3,4,5};
```

```
System.out.println(numbers[0]);
```

```
System.out.println(numbers[4]);
```

```
System.out.println(numbers.length);
```

5

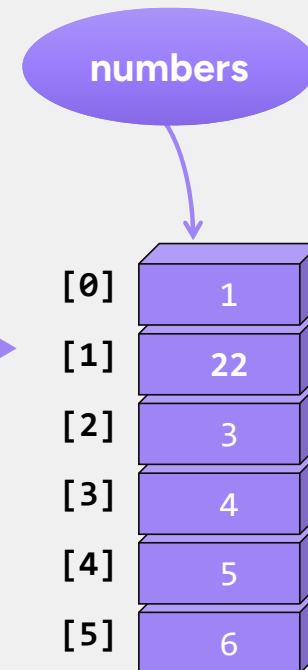


Introduction to arrays (2)

- You can change array elements.
- Appending, inserting, and removing an element is hard.

```
int[] numbers = {1,2,3,4,5};  
  
numbers[1] = 22;  
  
System.out.println(numbers[2]);
```

22



Create an empty array

```
int[] numbers = new int[5];
```

0
0
0
0
0

Initialised to default values
for int

```
String[] names = new String[5];
```

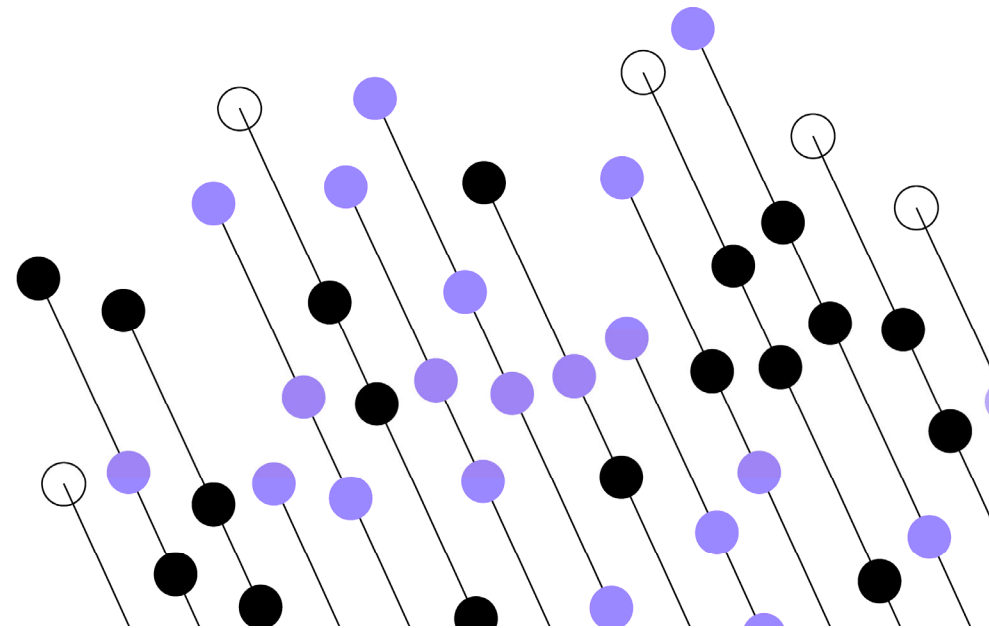
null
null
null
null
null

Initialised to default values
for String

Loops

Objective:

- To cover Java's looping constructs.



Iteration using WHILE and FOR statements

```
while ( boolean_expression ) {  
    statement(s);  
}
```

```
do {  
    statement(s);  
} while ( boolean_expression );
```

```
for ( init_expr; boolean_expr; update_expr ) {  
    statement(s);  
}
```

We'll see iteration using enhanced for loops later.

Iteration using WHILE loops

Declare/Init

```
int i = 0;
while (i < 10)
{
    System.out.printf("%d squared = %d\n", i, i*i );
    i++;
}
```

Exit condition

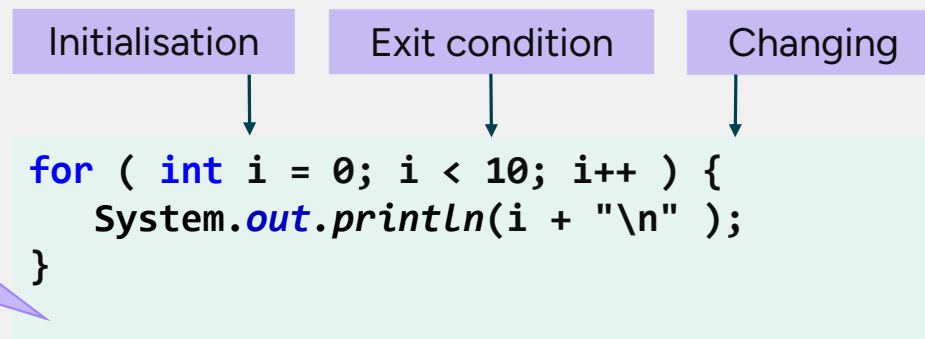
Changing value

...or put the test condition at the end of the loop:

```
int i = 0;
do
{
    System.out.printf("%d squared = %d\n", i, i*i );
    i++;
} while (i < 10);
```


The FOR loop

- Initialisation can include a declaration.
- Declared variable is in scope only inside the loop.



What is the value of 'i' here?

Initialisation and updates can be a list of ',' separated expressions.

```
for( int i = 0, j = 10; i < j; i++, j-- ) {  
    System.out.println( i * j + "\\n");  
}
```

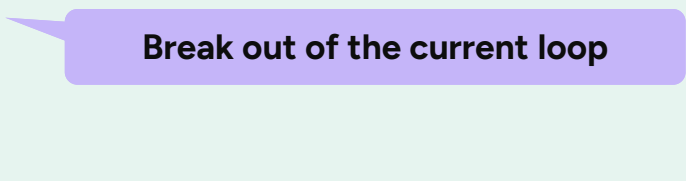
Using break to exit any loop

```
double money = 50;    // £ pounds
double interest = 0.06;    // percent

for (int years = 1; money < 1000; years++) {
    money += (money * interest);
    print("Year : "+ years + ": " + money);

    double tax = money * 0.40;
    if (tax > 100) {
        print("Tax is > 100");
        break;
    }
}

// code...
```



```
Year 1: 53
Year 2: 56
Year 3: 59
Year 4: 63
Year 5: 66
Year 6: 70
Year 7: 75
Year 8: 79
Year 9: 84
Year 10: 89
Year 11: 94
Year 12: 100
Year 13: 106
Year 14: 113
Year 15: 119
Year 16: 127
Year 17: 134
Year 18: 142
Year 19: 151
Year 20: 160
Year 21: 169
Year 22: 180
Year 23: 190
Year 24: 202
Year 25: 214
Year 26: 227
Year 27: 241
Year 28: 255
Tax is > 100
```

Using break to exit any loop — using label

```
outer_loop:
for ( ; ; ) {
    for ( ; ; ) {
        if (...) {
            break outer_loop;
        }
    }
}
...
```

Break out of nested loops



Continue

```
for ( ; ; )  
{  
    ...  
    ...  
    continue;  
    ...  
}  
...
```

A purple arrow originates from the 'continue;' statement and points to the closing brace of the for loop, indicating that the rest of the loop body is skipped.

```
for ( int i = 0; i < 10; i++ ) {  
    if ( i % 4 == 0 ) {  
        // few statements  
        continue;  
    }  
    // many statements  
}
```

```
for ( int i = 0; i < 10; i++ ) {  
    if ( i % 4 == 0 ) {  
        // few statements  
    }  
    else {  
        // many statements  
    }  
    // no code here!!  
}
```

Can be
coded as

The enhanced FOR loop

- For iterating over a collection or an array without testing for the bounds:

```
public void processNames( String[] names ) {  
    for (String name : names ) {  
        System.out.println( name );  
    }  
}
```

Read as: foreach String 'name'
in the 'names' collection

```
String[] names = {"Bob","Sasha"};  
for (String name : names) {  
    name += "x";  
}
```

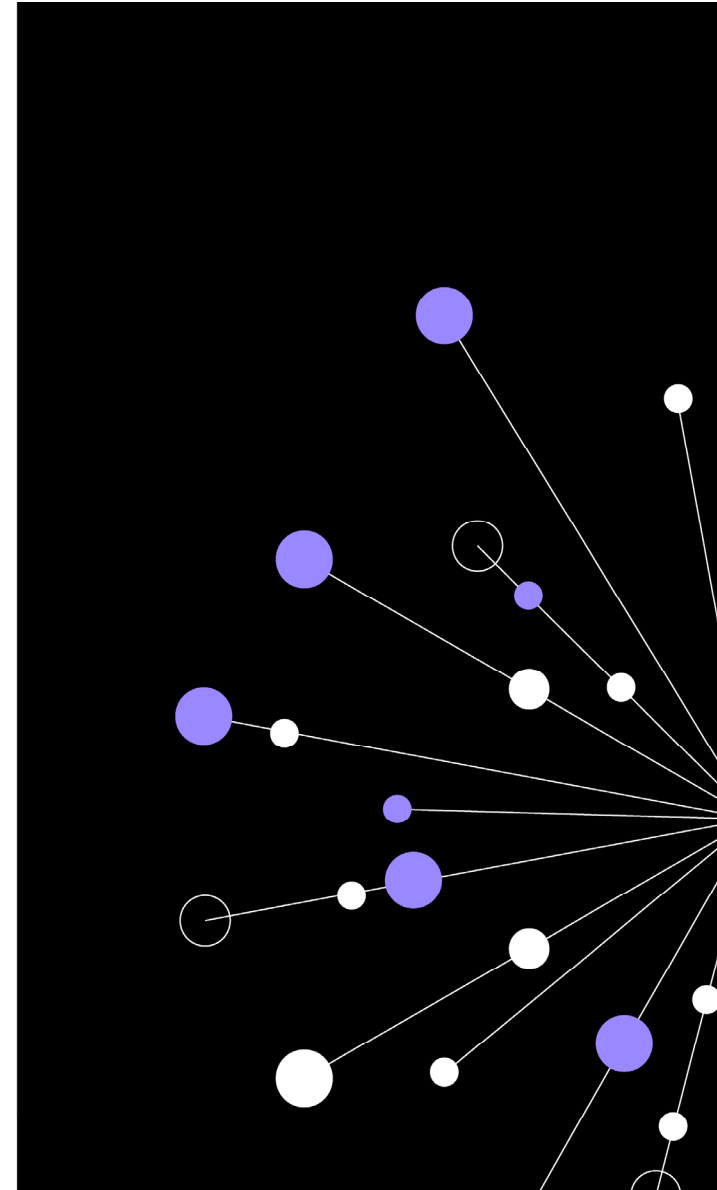
The elements are considered
read-only

```
System.out.println(names[0]);
```

Bob

In this chapter, we covered...

- Creating and using arrays
- Iteration statements:
 - WHILE
 - DO WHILE
 - FOR
 - Enhanced FOR loops



Lab

Objectives:

- Practice declaring and using arrays.
- Implement iteration using FOR and WHILE loops.
- Use the Java Enhanced FOR loop.
- Observe how arrays are passed by-reference.

Duration: 90 minutes.

Note: You will use GitHub to store your solutions and code examples. Detailed instructions will be provided in the lab's instructions.

